

Exam Strategy / Do Not Overlearn

- MAQ: mark only options you can justify by concept + assumption.
- Absolutes: always/never/cannot/only/must = suspect unless definition-level.
- Boundary check: mechanism? property? attacker? assumption? user behaviour?
- Do not overlearn: no sketching; no brainstorming phase lists; no exact angles/frequencies; no t-test calculations; no Fitts' equation.
- Learn: concepts, pros/cons, method choice, UCD application, scenario reasoning.

Core Chians
UCD: users/context → data gathering → themes → reqs → personas/scenarios → alternatives → lo/hi-fi prototype → evaluation → iterate.
Security analysis: asset → attacker/capability → threat → vuln → CIA impact → control → residual risk/usability cost.
Usable security: security benefit + user burden + likely workaround + safer default + clear feed-back/recovery.

Usability Goals		
Goal	Meaning	Sec angle
Effect	complete accurately	secure task succeeds
Efficiency	low effort/time	friction acceptable
Safety	avoid/recover errors	no mis-send/leak
Utility	right functions	real controls
Learnability	new user succeeds	warnings/MFA clear
Memorability	return easily	secure flow remembered

Human Factors / Accessibility		
Attention/perception/memory		warn-ings/labels/steps.
able,	operable,	understandable, robust.
Concept	Exam handling	Trap
Fitts	D/W = target geometry; a, b vary by device/user/sys	memorise equation
Recall/recognition	CLI recalls commands; menus/GUI recognise	CLI easier for all
Interact style	cmd: instruct; Q&A: converse; object: manip.; browse: explore; prompt: respond	CLI = style
Access	perm/temp/sit: motor, cog, stress, noise, mobile	colour, memory, jargon only

Evaluation Method Selection		
Method	Best when	Trap/clues
Usability test	users do tasks	task success; not sur-vey/expert review
Think-aloud	mental model, why	verbalise; do not teach/lead
Experiment	compare variants	IV/DV; no t-test calc
Field study	real context	natural use; less control
Heuristic eval	expert principles	no users; not task proof
Cognitive walkthrough	first-use steps	discover; assumptions
Inspection	checklist review	fast; can miss real users
Analytics/models	logs/patterns	logs do not explain motives

Evaluation Templates
Usability eval: goal; participants; tasks; method; data (success/time/errors/comments); procedure/ethics; findings; redesign.
Think-aloud: realistic task + ask to speak + prompt “keep talking” + observe; no teaching.
Experiment: hypothesis; IV; DV; controls; participants; within/between; counterbalance; validity.
Concrete task: tell goal not steps. Good: find

yesterday's step count / last month's avg steps. Bad: click History then Yesterday. Vague: explore steps.

Prototype / Design	
Lo-fi	Hi-fi
Use	ideas/flow/concept
Cost	cheap/changeable
Tests	task path, metaphor, layout
Sec use	permission concept
Trap	rough ≠ bad

Persona = evidence-based user type with goals/tasks/pain points. Scenario = user pursuing goal in context. Requirement ≠ feature wish.

Data Gathering
Interviews: open = exploratory; structured = consistent; semi-structured = planned+follow-up. Thematic analysis: notes → codes → themes → reqs/features. Recruit target users: USyd UI → USyd volunteers; campus bystanders ≠ necessarily students.
Boundary Correct use Trap
Early reqs ask goals, context, practice Qs solution-led feature
Known alts ask easier/preferred + leading all preference Qs = leading

CIA + Security Vocabulary		
Term	Meaning	Trap
Confid.	no unauth disclosure	not correctness
Integrity	no unauth change/tamper	encryption alone not enough
Avail.	service usable by authorised users	DoS is security
Threat	potential cause/attacker/event	not same as weakness
Vuln	exploitable weakness	not just bad outcome
Risk	likelihood × impact	needs context

Password Storage / Hashes
Hash = one-way verify; encryption = reversible. Store salted slow hashes; never plaintext. TLS protects transit; MFA helps after theft.

Item	Stored/secret	Purpose/trap
Salt	DB; not secret	per-user anti-rainbow; weak pw still guessable
Pepper	outside DB; secret	extra DB-breach protection; do not store by hash
Weak accts users × weak%		don't use total if targeting weak group
Worst/exp. accts × dict; exp. ≈ accts × dict/2		last vs average guess

Rainbow Tables
Precompute hash/reduction chains; storage-for-time tradeoff for likely passwords. Useful when hash setup/keyspace reusable. Salt breaks reuse across users. Not examinable code; know concept.

Crypto Boundaries		
Boundary	Left	Right/trap
Hash/encrypt	one-way verify	reversible confidentiality
Sym/asym	shared secret; fast	public/private; identity issue
PKE/signature	encrypt to pubkey	sign with private key
MAC/signature	shared secret	public verification
TLS/E2EE	auth client-server channel	endpoint content; server may see
Authn/authz	prove identity	TLS plaintext permission decision

Symmetric / OTP
Symmetric encryption: same secret key; efficient bulk confidentiality. Does not solve key distribution/auth/integrity. OTP perfect secrecy iff uniformly random, msg-length, secret, one-time key. Uniform-looking ≠ unpredictable: small deterministic state can be learned. “condition if violated” = positive req; “bad situation” = reuse/not random. **Trap:** encryption alone not auth.

Public Key / DH / MITM
PKE: public key encrypts; private key de-crypts; costlier than SE; use SE/AE for bulk data, PKE/DH/certs for key exchange, identity, small data. Need certs/fingerprints/trust. DH: derive shared secret; unauth DH → MITM. Bad/non-primitive generator → small subgroup/subgroup confinement; not same as computing private keys; not same as MITM. **Trap:** DH alone does not authentic.

DH item	Meaning	Trap
Hardness	discrete log; given $g, p, A = g^a$, find a	normal log/division fails
Secret	$A = g^a, B = g^b$, shared $= B^a = A^b = g^{ab} \bmod p$	not g^{a+b}
Toy recovery	small p : brute-force a until $g^a \equiv A$	Fermat not direct solve

MAC / HMAC / Signature / AE
MAC/HMAC: shared secret verifies integrity + authenticity among holders; verifier can also create MAC. Digital signature: private signs, public verifies; public verifiability. Authenticated encryption: confidentiality + integrity; reject tampered cipher-text before plaintext use. **Trap:** encryption-only ≠ integrity; MAC ≠ signature.

Authentication Stack
Identification = claim identity; authentication = verify; authorization = allowed action; account-ability = trace action. Password/token/biometric = factors; MFA combines know/have/are. MFA helps stolen password risk; does not eliminate phishing/push fatigue/recovery issues. Biometrics not easily changed; accessibility/privacy concerns. **Trap:** logged-in ≠ authorised.

TLS / HTTPS / Certs
TLS: server authentication via certificate + key exchange + symmetric session protection. HTTPS = HTTP over TLS. Protects network channel from eavesdrop/MITM if cert validation correct. Does not protect from malicious server, endpoint malware, wrong recipient, or server seeing plaintext. Cert validation: chain, host, validity, trust anchor. Pinning/hardcoded CA: reduces some MITM but update/recovery risk. **Trap:** accepting self-signed/disabled validation kills TLS guarantee.

TLS part	Role	Trap
Handshake	server auth + key negotiation before secrets	not bulk app transfer
Record	fragment + encrypt + integrity for app data	not cert validation
Heartbeat	keep-alive/liveness	Heartbleed abused length
Server auth	CA cert + chain, host, date, trust + private-key proof	not login auth

E2EE
E2EE encrypts at sender endpoint, decrypts at recipient endpoint; server/intermediary should not read content. Does not hide all metadata, fix compromised endpoints, verify identity automatically, or stop forwarding screenshots/plaintext. Usability: key verification/device-change warnings must be understandable and rare enough to matter.

Attack Map			
Attack	Cause	CIA	Mitigation
SQLi	input becomes SQL	C/I/A	prepared stmts; least privilege
XSS	input rendered as script	C/I	output encoding, sanitise, CSP
Inference	combine aggregates	C/privacy	views, noise, overlap limits, audit
SYN flood	half-open TCP table	A	SYN cookies; timeouts; filtering
DoS/DDoS	resource exhaustion	A	scale; filter; rate limit
Heartbleed	unchecked heartbeat length	C	patch, bounds check, rotate keys

CSRF: not confirmed focus in provided materials; background only.

Inference Attack Pattern
Attacker is legitimate aggregate user + prior knowledge + repeated/overlapping accepted queries. Small-result rejection alone insufficient. Example: accepted sums over sets differing by target → subtract to infer target salary. Mitigate: safe views/granularity, query auditing, noise/partial disclosure, overlap limits.

SQLi vs XSS
SQLi executes in DB query context; often server-side data impact. XSS executes in browser/page context; steals tokens, acts as user, changes content. SQLi best answer: parameterised queries. XSS best answer: context-aware output encoding + sanitisation/CSP. **Trap:** “use HTTPS” does not fix SQLi/XSS.

Input	Control	Trap
HTML/script text	encode; sanitise; CSP	raw render
SQL-like text	parameterised queries	HTTPS fixes
File upload	type/size validate; scan; isolate	ordinary text sanitise enough

Network / Software Bugs
SYN flood: TCP handshake abuse; many SYNs no completion; availability. Heartbleed: memory disclosure from trusting length; confidentiality; may expose private key/tokens/password fragments. Static analysis: code, no run; false positives. Dynamic: run with inputs; misses untested paths. Symbolic: path constraints; path explosion. Tools need actionable developer UX. **Layers:** routers/routing/path/IP/across networks = network; TCP/UDP/ports/SYN/ACK/reliability = transport; HTTP(S)/browser/web req/app protocol = application; Wi-Fi/Ethernet/MAC frame = link.

Protocol/analysis	Role/associated	Trap
IP	address + route; best-effort	ordered reliable delivery
TCP	unreliable seq, ACK, retry, reasm.; reliable ordered	encrypts contents
UDP	no built-in delivery/order/retry	guarantees packets
Static	AST/CFG/code no run	runtime logging
Dynamic	path runtime behaviour, leaks, UAF	syntax-only
Symbolic	path constraints; symbolic inputs	ignores path explosion/env model

Usable Security Rules
Secure defaults; least privilege; clear permission state; specific warning risk + safe action; usable recovery; avoid warning fatigue; ordinary users should not manage raw crypto; treat insecure workload as design evidence; privacy/convenience trade-off explicitly.

Warning Answer Mini-Template
Risk? user decision? language specific? safe action clear? frequency/habituation? fallback/recovery? likely behaviour under stress? residual risk?

Project-Reusable Patterns
Group-work system: requirements gathering, themes, feature justification, lo-fi/hi-fi prototypes, think-aloud, auth, password hashing/salting, TLS/cert validation, E2EE, vuln demo. Justify design: user theme + security threat + chosen control + usability impact.

Tutorial Transformations

W Skill	Exam move
2 usability goals	classify issue; propose fix
3 interviews/themes	data → theme → req
4 persona/scenario	choose design from evidence
5 think-aloud	plan tasks; avoid leading
7 OTP/hash/key reuse	assumptions fail?
8 PKE/MAC/sign/DH	match goal to mechanism
9 rainbow/TLS	salt/cert/E2EE boundary
10 inference/SQLi/HTTPS	threat + layer + mitigation
11 XSS, SYN, Heartbleed	classify root cause/CIA

Scenario Answer Stems
Usability: user + goal + task barrier + affected goal + evidence/method + redesign.
Security: asset + attacker + vuln + CIA + assumptions + mitigation + usability cost.
Trade-off: security gain vs user burden; likely workload; safer default; residual risk.
Method choice: why this method fits clue words; why alternatives weaker.

Common Wrong Answers
“Looks nice” = usability; “encrypt passwords”; “HTTPS = E2EE”; “authn = authz”; “salt secret”; “MFA stops all phishing”; “DH authenticates”; “expert eval = user test”; “field study always better”; “logs explain why”; “aggregate data always safe”; “all input attacks are same”.

One-Line Examples
Cert warning clicked through → think-aloud for mental model + better warning.
Wrong recipient file share → usability safety + confidentiality.
Stolen password DB → salt + slow hash + MFA + TLS.
Self-signed cert accepted → MITM despite HTTPS UI.
Repeated MFA prompts approved → prompt fatigue usable-security failure.
Aggregate salary queries overlap → inference attack.

Final MAQ Scan
For each option: definition true? assumptions stated? property correct? method fits clue words? mechanism solves that problem? usability/security side ignored? If unsure under negative marking, leave blank.

Security Boundary Table

Boundary	Distinction	Exam trap
Hash/encrypt	hash one-way verify; encrypt reversible w/key	“decrypt hash”
TLS/E2EE	TLS channel to server; E2EE endpoint content	HTTPS hides from server
Authn/authz	verify identity; decide	logged in = allowed
Ident/authn	permission claim name; prove claim	username = proof
MAC/sig	shared secret; private/public proof	MAC public-verifiable
Sym/asym	one shared key; keypair	ignore key distribution
DH/auth DH	shared secret; also	plain DH stops MITM
Salt/key	identity-bound salt public	salt must be secret
C/I	per-password; key secret	secret
C/A	secrecy; tam-per/correctness disclosure; service access	encryption = integrity DoS steals data
Threat/vuln/risk	cause; weakness; likelihood-impact	impact = vuln
DoS/theft	availability loss; confidentiality loss	all attacks steal data
Cert val/encrypt	identity binding; secrecy mechanism	encrypt w/wrong server ok
PW hash/TLS	storage protection; transit protection	one replaces other
MFA/phishing	extra factor; not phishing-proof	approve-prompt safe
AE/encrypt-only	C+I; C only	safe ciphertext
PKE/SE bulk	PKE costly; SE/AE bulk	tamper ignored use PKE for big files

Evaluation Boundary Table

Boundary	Choose when	Trap
User test/heuristic	real users/tasks vs expert principles	expert review = user evidence
Think-aloud/interview	task cognition vs reported experience	teaching during TA
Experiment/user test	causal issues	DV as IV compare vs find task
Field/lab	natural context vs control	field always better
Cog walkthrough/heuristic	first-use steps vs broad principles	just browse app
Inspection/user test	expert check vs behaviour evidence	no users = proved usable
Analytics/user study	log patterns vs why/mental model	logs explain motives
Qual/quant	themes vs measures	qualitative = weak
Formative/summative	improve only test at end	only test at end
Lo-fi/hi-fi	design vs final judgement	early concept vs realistic detail

Attack / Vulnerability Boundary

Issue	Cause + property	Mitigation/trap
SQLi	SQL concat; C/I/A	parameterise; HTTPS not fix
XSS	unescaped output; C/I aggregate	encode; sanitise; CSP
Inference	overlap; C/privacy resource	restrict/noise; rows hidden not enough
SYN/DoS	exhaustion; A bounds bug; C	filter/rate-limit; not theft
Heartbleed		patch; rotate keys; not DoS
Phishing/social	deceived user; auth/C/I	MFA + UX; MFA not magic
Cert fail	no identity check; C/I	validate
OTP reuse	pad reused; C	chain/host/trust never reuse; OTP conditions
Weak PW store	fast/plain hash; C/auth	salt + slow hash
Unauth DH	active MITM; C/I	auth key exchange
Warn fatigue	vague frequent alerts; user error	rare actionable warnings
Token theft	session secret leak; auth/C/I	cookie flags; fix XSS

UCD / Project / Research Boundaries

Boundary	Correct distinction	Common mistake
Goal/task	desired outcome vs action	task list as user goal
Theme/req	sequence data	theme = feature
Req/feature	pattern vs system need	feature = no
Persona/profile	need vs implementation idea	“cool feature”
Scenario/task list	goals/tasks vs demographics	evidence age/name only
Obs/interp	narrative in context vs steps only	no context/text/goal
Func/non-func	what happened vs meaning system	mix fact with inference
Interview/survey	does vs quality constraint	survey gives why
Proto feedback/correctness	broad Qs	user prototype proves
Finding/recommendation	response vs code works	implementation jump without evidence

Experiment Traps

Term	Meaning / trap
IV	manipulated factor (e.g., warning design)
DV	measured outcome (e.g., correct choice/time)
Control var	held constant
Confound	alternative explanation
Within-subj	same participants all conditions; order risk
Between-subj	different groups; group differences risk
Counterbalance	controls order/learning effects
Hypothesis	one clear comparison + one main DV; split speed and accuracy
Internal val.	causal confidence
External val.	generalisability
t-test	know purpose only; no calculation

Accessibility / Situational Impairment

Permanent/temporary/situational impairments: visual, hearing, motor, cognitive, stress, noise, mobile, time pressure. Accessible design helps beyond disabled users. Security UI must be accessible: MFA, warnings, CAPTCHA, recovery, permission state. Avoid colour-only signals, memory-only flows, tiny targets, fine motor actions, jargon.

Security Warning Traps

Good warning = rare + specific + consequence clear + safe action clear. Bad warning =

vague/frequent/technical/dismissible. False positives reduce trust. Safe default easier than risky override. Time pressure ⇒ click-through. Habituation = repeated warning ignored.

Authentication Factors

Factor	Notes
Know	password/PIN; guess/phish/reuse risk
Have	phone/token/security key; loss/recovery risk
Are	biometric; cannot rotate like pw; privacy/accessibility
MFA	combine independent factors; recovery weakest link
Session token	not password, but bearer secret; protect from XSS/leak

TLS Key Exchange / X25519

Modern TLS derives session keys via authenticated key exchange. X25519/ECDHE = key agreement, not bulk encryption. Bulk data uses symmetric AE (e.g., AES-GCM/ChaCha20-Poly1305). Forward secrecy = past sessions safe if long-term key later leaks. Cert validation binds server identity to key. Disable validation ⇒ MITM.

User Study Ethics

Consent; purpose; voluntary participation; privacy; anonymisation; avoid harm; collect only needed data; allow withdrawal; secure notes/transcripts; separate observation from interpretation. Human participants + research data + publication → ethics approval; still if anonymised/no audio-video. Anon lowers risk, not requirement.

Req / Feature Justification

Functional req = what system does. Non-functional req = quality/security/usability constraint. Strong chain: evidence/theme → user goal → requirement → feature → benefit → evaluation method. Weak: feature because modern/cool/AI-generated/pretty.

Failure Pattern Map

Failure	Reasoning / fix
Weak PW store	stolen DB → offline guessing → salt+slow hash
Cert val off	MITM → validate chain/host/trust
OTP reuse	ciphertext relation leak → never reuse pad
Unauth DH	MITM → authenticate exchange
Bad DH generator	small subgroup risk → safe params
SQL concat	SQLi → parameterised queries
Unescaped output	XSS → contextual encoding/CSP
Aggregate overlap	inference → audit/noise/restrict
Vague warnings	habituation → rare/actionable alerts
Strict security	workaround → usable secure defaults

Scenario Mini-Templates

Usability eval: goal → users → task → method → data/metric → finding → design change.

Security: asset → attacker → threat → vuln → CIA → mitigation → assumption → residual risk.

Trade-off: security benefit → user burden → likely workaround → reduce burden → residual risk.

Project/report: interview evidence → theme → requirement → feature → risk/control → evaluation evidence.

MAQ check: true under stated assumptions? always/sometimes? mechanism vs goal? scenario-specific?

Clue Words

“Why did user choose?” = think-aloud/interview. “Can users complete task?” = usability test. “Which design causes better outcome?” = experiment. “Real-world context?” = field. “Expert/no users/quick?” = heuristic/inspection. “First-time discover?” = cognitive walkthrough. “Logs/frequency?” = analytics.

High-Yield Last Checks

Security control must match threat. Usability issue must match user/task/context. Every mitigation has assumptions and residual risk. Every security feature has user cost. Negative marking: select only if boundary is clear.